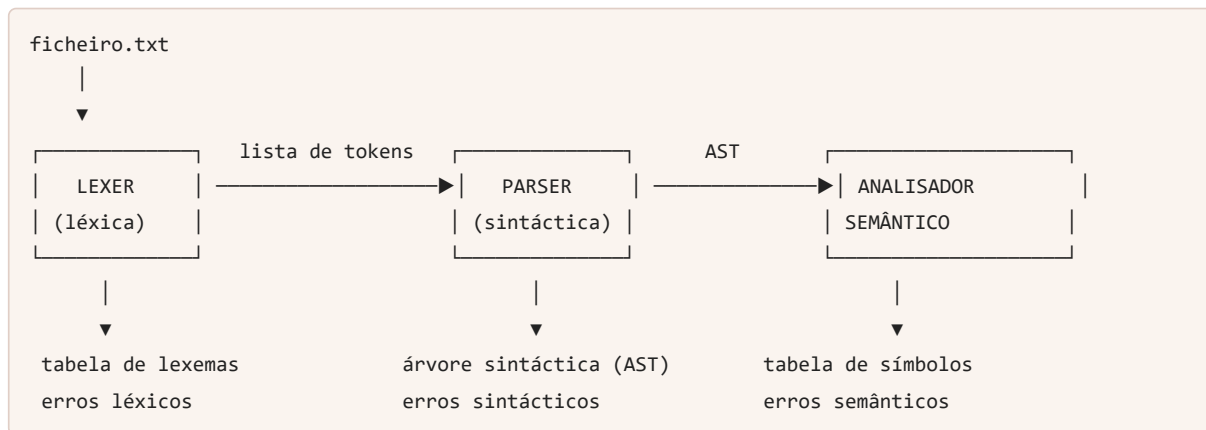


Explicação do Analisador — fase a fase

Guia de estudo do projeto: o que cada ficheiro principal faz, por que ordem as fases correm, e como a árvore sintáctica (AST) é construída e desenhada.

Visão geral do fluxo



Os três analisadores partilham um único colector de erros (`src/erros/ColetorErros.java`): nenhuma fase aborta no primeiro erro — todos os erros são acumulados e mostrados no fim, ordenados por linha.

FASE 1 — Análise Léxica (`src/lexico/Lexer.java`)

O que faz: transforma o texto do ficheiro numa **lista de tokens**.

Como funciona: o método `analisar()` percorre o código **carácter a carácter** (nunca usa regex nem geradores) e decide o que fazer olhando para o carácter actual:

- Espaço, tab, quebra de linha → ignora (mas `avancar()` conta as linhas, para os erros terem a localização certa).
- `//` ou `/* */` → salta o comentário inteiro.
- Letra → lê um identificador (`lerIdentificador()`); se o texto estiver no conjunto `PALAVRAS_RESERVADAS` (`switch`, `case`, `default`, `break`, `int`, `String`, `final`), classifica como palavra reservada, senão como `IDENTIFICADOR`.
- Dígito (ou `-` seguido de dígito) → `LITERAL_INT`.
- `"` → lê até às próximas aspas (`lerString()`); se a linha acabar antes, regista o erro "string não terminada".
- `=` `(` `)` `{` `}` `:` `;` → token de símbolo.
- Qualquer outro carácter → **erro léxico** "carácter inválido", e o lexer salta-o e continua (recuperação).

Saída: `List<Token>` — cada `Token` (`src/lexico/Token.java`) guarda o **lexema** (texto), a **classe** (`src/lexico/ClasseToken.java`), a **linha** e a **coluna**. No fim da lista vai um token sentinela `EOF`, que evita que o parser tenha de testar "acabou a lista?" a cada passo.

Lista de classes de token:

Classe	Lexemas
PALAVRA_RESERVADA	switch case default break int String final
IDENTIFICADOR	letra seguida de letras/dígitos
LITERAL_INT	dígitos, com - opcional
LITERAL_STRING	"texto"
OP_ATRIBUICAO	=
DELIMITADOR	() { }
PONTUACAO	: ;

FASE 2 — Análise Sintáctica (`src/sintatico/Parser.java`)

O que faz: verifica se a **ordem** dos tokens obedece à gramática e, ao mesmo tempo, **constrói a AST**.

A gramática (notação BNF — Forma de Backus-Naur)

Na notação usada nas aulas: não-terminais entre `< >`, `::=`, cada alternativa como regra numerada, `ε` para vazio, e a gramática como quádruplo `G = (Vn, Vt, P, S)`.

```

P:
1. <programa>      ::= <lista_decl> <lista_switch>
2. <lista_decl>    ::= <declaracao> ; <lista_decl>
3. <lista_decl>    ::= ε
4. <declaracao>    ::= <tipo> ident
5. <declaracao>    ::= <tipo> ident = <valor>
6. <declaracao>    ::= final <tipo> ident = <valor>
7. <tipo>          ::= int
8. <tipo>          ::= String
9. <lista_switch>  ::= <switch> <lista_switch>
10. <lista_switch> ::= <switch>
11. <switch>       ::= switch ( <selector> ) { <lista_caso> }
12. <switch>       ::= switch ( <selector> ) { <lista_caso> <default> }
13. <selector>     ::= ident
14. <lista_caso>   ::= <caso> <lista_caso>
15. <lista_caso>   ::= <caso>
16. <caso>         ::= case <constante> : <lista_instr>
17. <default>      ::= default : <lista_instr>
18. <lista_instr>  ::= <instrucao> <lista_instr>
19. <lista_instr>  ::= ε
20. <instrucao>    ::= <switch>
21. <instrucao>    ::= <atribuicao>
22. <instrucao>    ::= <break>
23. <atribuicao>    ::= ident = <valor> ;
24. <break>        ::= break ;
25. <valor>        ::= <constante>
26. <valor>        ::= ident
27. <constante>    ::= lit_int
28. <constante>    ::= lit_string

G = ({<programa>, <lista_decl>, <declaracao>, <tipo>, <lista_switch>,
    <switch>, <selector>, <lista_caso>, <caso>, <default>, <lista_instr>,
    <instrucao>, <atribuicao>, <break>, <valor>, <constante>},
    {switch, case, default, break, int, String, final,
    ident, lit_int, lit_string, =, (, ), {, }, :, ;},
    P, <programa>)

```

- Regra 20 (<instrucao> ::= <switch>) → **aninhamento** sem limite.
- Regra 9 (<lista_switch> recursiva) → switches em **cadeia**.

Que tipo de gramática é esta?

É uma **gramática livre de contexto** (GLC) — **Tipo 2 na hierarquia de Chomsky**: todas as produções têm um único não-terminal à esquerda (<A> ::= α). É o tipo usado para a sintaxe de linguagens de programação porque expressa estruturas **recursivas/aninhadas** (switch dentro de switch), o que uma gramática regular (Tipo 3) não consegue. Os tokens em si (ident , lit_int) são regulares — por isso ficam a cargo do lexer, e a GLC trata só da estrutura.

Além disso a gramática é **LL(1)**:

- **LL** — lê a entrada da esquerda para a direita (*Left-to-right*), construindo a derivação mais à esquerda (*Leftmost*);
- **(1)** — basta **1 token de lookahead** para escolher a produção: se o token é switch , final , case ou break , cada um determina uma regra diferente, sem ambiguidade nem backtracking.

É exactamente essa propriedade que permite implementar a gramática como **parser descendente recursivo** — cada não-terminal vira um método e um `if` sobre o token actual escolhe a produção. Se houvesse recursão à esquerda (`<A> ::= <A> x`) ou duas produções começadas pelo mesmo token, LL(1) falharia e seria preciso transformar a gramática ou usar um parser LR (tabular).

Dava para simplificar a gramática?

Em BNF pura (a notação das aulas): pouco. BNF pura não tem `|` na mesma linha nem `[ ]` opcional — cada alternativa tem de ser uma regra própria, e é por isso que são 28. Podia-se raspar 3–4 regras (eliminar `<selector>` e `<constante>`, fundir as regras 11/12 com um `<default> ::= ε`), mas isso piora a legibilidade: esses não-terminais existem para dar nome aos conceitos do enunciado.

Em EBNF: sim, encolhe para metade. Com `|`, `[ ]` (opcional) e `{ }` (repetição), as 28 regras viram ~10 linhas:

```
<programa> ::= { <declaracao> ; } <switch> { <switch> }
<declaracao> ::= [ final ] <tipo> ident [ = <valor> ]
<switch> ::= switch ( ident ) { <caso> { <caso> } [ <default> ] }
```

Mantivemos a **BNF pura com listas recursivas** porque é a notação dos apontamentos (F5): "listas sintácticas são descritas usando recursão" — as regras 2, 9, 14 e 18 são exactamente essas listas. Resposta pronta para a defesa: "dava para compactar em EBNF, mas mantivemos BNF pura com recursão, como nas aulas."

Parser descendente recursivo

Cada não-terminal da gramática é um método Java com o mesmo nome:

Regra da gramática	Método no Parser
PROGRAMA	analisar()
LISTA_DECL	parseListaDecl()
DECLARACAO	parseDeclaracao()
LISTA_SWITCH	parseListaSwitch()
SWITCH	parseSwitch()
CASO / DEFAULT	parseCaso() / parseDefault()
LISTA_INSTR	parseListaInstr()
ATRIBUICAO	parseBreak() / parseAtribuicao()
VALOR	parseValor()

O parser olha para o **token actual** (lookahead de 1) para decidir a regra: se vê `switch`, chama `parseSwitch()`; se vê um identificador dentro de um case, chama `parseAtribuicao()`; se vê `break`, `parseBreak()`.

- **Cadeia** de switches: o `while (verificaLexema("switch"))` em `parseListaSwitch()` lê um switch atrás do outro.
- **Aninhamento**: dentro de `parseListaInstr()`, se o token é `switch`, chama-se `parseSwitch()` **recursivamente** — é assim que um switch dentro de um case funciona, sem limite de profundidade.

Recuperação de erros

Quando falta um token esperado, `exigirLexema()` / `exigirClasse()` registam o erro ("esperado X mas encontrado Y", com a linha) e o método `sincronizar()` salta tokens até um ponto seguro (`;`, `case`, `default`, `}`, `switch`). Assim um único erro não gera uma cascata e o resto do ficheiro é analisado.

A ÁRVORE SINTÁCTICA — AST (`src/sintatico/*.java`)

A AST é o **resultado** do parser: uma árvore de objectos Java onde cada nó é uma construção da linguagem. Os ficheiros dos nós:

Ficheiro	Nó que representa
<code>Programa.java</code>	a raiz: lista de declarações + lista de switches
<code>Declaracao.java</code>	<code>int x = 1;</code> ou <code>final int K = 5;</code>
<code>ComandoSwitch.java</code>	um switch: selector + lista de casos + default
<code>Caso.java</code>	um case (rótulo + instruções) ou o default
<code>Instrucao.java</code>	classe base abstracta das instruções
<code>Atribuicao.java</code>	<code>x = valor;</code> (destino, valor, classe do valor)
<code>ComandoBreak.java</code>	<code>break;</code>

O truque do aninhamento: `ComandoSwitch` extends `Instrucao`. Como um caso guarda `List<Instrucao>`, um switch pode estar dentro de um case tal como uma atribuição ou um break — a própria estrutura de classes espelha a gramática.

Como a árvore é construída

Cada método `parseXxx` devolve o nó correspondente, e o método que o chamou liga-o como filho:

```
// dentro de parseSwitch():
ComandoSwitch comando = new ComandoSwitch(selector, ...);
while (verificaLexema("case")) {
    comando.casos.add(parseCaso()); // cada caso vira filho do switch
}
// dentro de parseListaInstr():
if (verificaLexema("switch")) {
    lista.add(parseSwitch()); // switch aninhado vira filho do case
}
```

As DUAS árvores: derivação vs. AST

O projecto mostra duas árvores na interface, e a diferença entre elas é matéria clássica de defesa. Para este código:

```
switch (opcao) {
    case 1: mensagem = "um"; break;
    default: mensagem = "outro";
}
```

1. Árvore de sintaxe / de derivação (aba "Arvore de sintaxe" — notação dos apontamentos F3): a raiz é o símbolo inicial e cada nó não-terminal expande **nos símbolos do lado direito da regra BNF aplicada**, com os terminais (incluindo `(`, `{`, `:`, `)` como folhas e `ε` nas regras vazias:

```

<programa>
├─ <lista_decl>
│   └─ ε
└─ <lista_switch>
    └─ <switch>
        └─ switch ( <selector> ) {
            └─ opcao
            └─ <lista_caso>
                └─ <caso>
                    └─ case <constante> :
                        └─ 1
                        └─ <lista_instr>
                            └─ <instrucao>
                                └─ <atribuicao>
                                    └─ mensagem = <valor> ;
                                        └─ <constante>
                                            └─ "um"
                            └─ <lista_instr>
                                └─ <instrucao>
                                    └─ <break> → break ;
                                └─ <lista_instr> → ε
                    └─ <default>
                        └─ default : <lista_instr> → ...
                └─ }
        └─ }

```

2. AST — árvore de sintaxe ABSTRACTA (aba "AST (simplificada)": a derivação **sem pontuação nem nós de lista** — só a estrutura. É o "*can become*" dos slides: `if-statement(IF cond THEN stmt)` *can become* `if-statement(cond, stmt)` :

```

PROGRAMA
├─ switch (opcao)
│   └─ case 1
│       └─ =
│           └─ mensagem
│               └─ "um"
│       └─ break
└─ default
    └─ =
        └─ mensagem
            └─ "outro"

```

Porquê as duas? A árvore de derivação prova que a sentença pertence à linguagem (cada nó cita uma regra da gramática); a AST é a que o compilador guarda em memória e que a análise semântica percorre — a pontuação já cumpriu o seu papel na fase sintáctica e é descartada. No código, a AST é a estrutura de objectos `Programa / ComandoSwitch / Caso / ...` e a árvore de derivação é reconstruída para visualização seguindo as regras numeradas (`InterfaceGrafica.noListaDecl`, `noSwitch`, `noCaso`, ... — cada método tem a regra que implementa como comentário).

FASE 3 — Análise Semântica (`src/semantico/AnalizadorSemantico.java`)

O que faz: percorre a AST (já construída) e verifica o **significado**, usando a **tabela de símbolos** (`src/tabelas/TabelaSimbolos.java`).

Corre em duas passagens:

Passagem 1 — declarações (`processarDeclaracao`): cada `Declaracao` da AST vira um `Simbolo` (identificador, categoria variável/constante, tipo, valor, linha) inserido na tabela. Se o nome já existir → erro "declaração duplicada".

Passagem 2 — switches (`verificarSwitch` , recursivo): para cada switch da AST verifica:

1. o **selector** está declarado? (procura na tabela)
2. o tipo do selector é `int` ou `String` ?
3. cada rótulo de `case` é compatível com o selector? (selector `int` → rótulos `LITERAL_INT` ; selector `String` → rótulos `LITERAL_STRING`)
4. em cada **atribuição**: o destino está declarado? o tipo do valor é igual ao tipo do destino? o destino é constante (`final`)? → erro
5. `case` duplicado no mesmo switch? (guarda os rótulos num `Set`)

Quando encontra um `ComandoSwitch` dentro de um case, chama `verificarSwitch` recursivamente — a verificação acompanha o aninhamento da árvore.

Como a árvore é DESENHADA (`src/InterfaceGrafica.java`)

A aba "Arvore (AST)" usa a classe interna `PainelArvoreAst` , que desenha o diagrama clássico (pai em cima, linhas em leque para os filhos):

1. **Conversão** — `preencherArvore(Programa)` converte a AST do parser em nós simples de desenho (`NoDesenho` = texto + filhos), com os lexemas como rótulos (`switch` (opcao) , `case 1` , `=` , `break`).
 2. **Layout** — `larguraSubarvore()` calcula recursivamente a largura de cada subárvore: `máx(largura do texto, soma das larguras dos filhos)` . É isto que impede os ramos de se sobreporem.
 3. **Desenho** — `desenhar()` pinta o nó centrado no seu intervalo horizontal e traça uma linha para o centro de cada filho, um nível (70 px) abaixo.
 4. **Zoom-to-fit** — antes de desenharmos, `paintComponent` compara o tamanho da árvore com o do painel e aplica `g2.scale(...)` para a árvore **cabem sempre inteira** no ecrã (nunca amplia acima de 1:1, só reduz).
-

Resumo: um ficheiro por responsabilidade

Responsabilidade	Ficheiro
Fase 1 — tokens	<code>src/lexico/Lexer.java</code>
Fase 2 — gramática + construção da AST	<code>src/sintatico/Parser.java</code>
Fase 3 — tipos e declarações	<code>src/semantico/AnalizadorSemantico.java</code>
Nós da AST	<code>src/sintatico/{Programa,Declaracao,ComandoSwitch,Caso,Instrucao,Atribuicao,ComandoBreak}.java</code>
Tabela de símbolos	<code>src/tabelas/TabelaSimbolos.java</code>
Colector de erros (partilhado)	<code>src/erros/ColetorErros.java</code>
Saída em consola (faseada)	<code>src/relatorio/Relatorio.java</code> + <code>src/Main.java</code>
Interface gráfica + desenho da árvore	<code>src/InterfaceGrafica.java</code>